

Image Captioning de Frames de Filmes com Modelos Multimodais

Trabalho Prático 3 — SGAIM

Miguel Alcobia
DEI
ISEL, IPL
Lisboa, Portugal
a50746@alunos.isel.pt

Fábio Pestana
DEI
ISEL, IPL
Lisboa, Portugal
a50756@alunos.isel.pt

Abstract—Neste trabalho desenvolvemos um sistema de *image captioning* para *frames* de filmes, com o objetivo de gerar legendas com as descrições a partir de imagens extraídas de filmes. Adotámos uma arquitetura multimodal composta por um *encoder* visual (CLIP), um módulo projetor e um *language model* (TinyLlama-1.1B). Avaliámos quatro configurações resultantes da combinação de dois *encoders* visuais (CLIP ViT-B/16 e ViT-B/32) com dois tipos de projetor (Linear e MLP). A melhor configuração, CLIP-16 com projetor MLP, atingiu BLEU-4 de 7.85%, ROUGE-1 de 41.51% e ROUGE-L de 28.01% num conjunto de teste composto por 112 *frames* de quatro filmes distintos.

I. INTRODUÇÃO E MOTIVAÇÃO

A capacidade de descrever automaticamente o conteúdo visual de uma imagem em linguagem natural é uma tarefa importante na interseção da visão computacional e do processamento de linguagem natural. O *image captioning* encontra aplicabilidade no dia a dia em sistemas de acessibilidade, organização de conteúdos e motores de pesquisa multimodais.

Neste trabalho, a modalidade escolhida foi a de imagem, e a tarefa concreta foi a geração de legendas descritivas a partir de *frames* extraídos de filmes. O *dataset* utilizado, MovieTecton_Mini, apresenta um desafio realista: as imagens possuem iluminação variável, posições da câmara, múltiplos personagens e cenários complexos, tornando-as substancialmente mais difíceis de descrever do que imagens simples e com muito poucos elementos, tipicamente utilizadas em exercícios e testes de *captioning*. O facto de também existirem no *dataset* filmes animados, contrasta com os restantes filmes realistas, o que aumenta ligeiramente a dificuldade.

A motivação para a escolha desta tarefa insere-se no âmbito do Trabalho Prático 3 da Unidade Curricular de Sistemas Generativos e Agentes Inteligentes Multimodais (SGAIM).

O grupo optou por explorar a geração de legendas para as imagens como forma de aplicar os conceitos lecionados num cenário com dados reais e mais complexos do que os dados em aula.

A combinação de *encoders* como o CLIP com modelos generativos como o TinyLlama cria uma abordagem que permite perceber de forma prática o impacto do *encoder* e do

módulo de projeção no desempenho final. O estudo sobre o processo de avaliação e procura de métricas para os resultados do modelo também contribuiu para a escolha deste tema para o projeto.

II. TRABALHOS RELACIONADOS

O problema de *image captioning* tem tido diferentes abordagens ao longo dos anos. Os primeiros trabalhos baseavam-se em CNNs para extrair *features* visuais, alimentando um *decoder* LSTM (*Long-Short Term Memory*) para geração de texto [1].

Com a introdução dos *transformers*, a abordagem passou a utilizar *encoders* visuais baseados em ViT (*Vision Transformer*). O CLIP (*Contrastive Language-Image Pre-training*) [2], desenvolvido pela OpenAI, propôs um pré-treino eficiente que aprende representações visuais diretamente de linguagem natural em grande escala. O modelo combina um *encoder* visual (como o ResNet ou ViT) e um *encoder* textual (*Transformer*) treinados simultaneamente para prever, num *batch* de 400 milhões de pares imagem-texto recolhidos da Internet, quais pares realmente correspondem um ao outro.

BLIP [3] (*Bootstrapping Language-Image Pre-training*) introduziu um mecanismo de *bootstrapping* (que gera e filtra legendas sintéticas) para lidar com o ruído dos dados *web*. Embora demonstre resultados superiores, o seu tamanho tornou-o computacionalmente proibitivo em recursos limitados. No presente trabalho, o BLIP foi considerado mas abandonado por instabilidade de treino nas máquinas dos membros do grupo por falta de recursos.

LLaVA [4] (*Large Language and Vision Assistant*) utiliza um *encoder* visual CLIP, um projetor linear simples e um LLM (Vicuna/LLaMA), sendo treinado em duas etapas: primeiro apenas o projetor (com o *encoder* e o LLM congelados) e, depois, o projetor e o LLM em conjunto para seguir instruções multimodais. A nossa implementação segue este paradigma, adaptado a recursos computacionais reduzidos com o TinyLlama-1.1B. No nosso projeto, deixou-se de utilizar o projetor simples e passou-se a usar um projetor com duas camadas e uma função de ativação (GELU) entre elas.

III. DATASET

A. Descrição

O *dataset* utilizado é o MovieTecton_Mini, disponível na plataforma Hugging Face (DIS-CO/MovieTecton_Mini) [5]. O *dataset* foi selecionado com o apoio do docente da cadeira, uma vez que o grupo achou interessante três *datasets*.

O *dataset* contém 560 *frames* de quatro filmes: 21 *Jump Street*, *Frozen*, *IF* e *The Fall Guy*, com 140 *frames* por filme. Cada amostra inclui: (i) o *frame* do filme (imagem PIL, *Python Imaging Library*); (ii) uma legenda descritiva do *frame*; (iii) o nome do filme; (iv) uma *label* binário (0 = unseen, 1 = seen) para a tarefa original de MovieTecton; e (v) uma lista com o nome do filme como *ground-truth*.

B. Pré-processamento

No *encoder* CLIP, as imagens são processadas com o CLIP-Processor que lhe é nativo, onde é feito um redimensionamento para 224×224 e a normalização. As *features* são extraídas uma única vez antes do treino e guardadas em memória, o que acelera o processo do treino.

As legendas foram tokenizadas com o *tokenizer* do TinyLlama, sendo acrescentado um token EOS no final de cada sequência e aplicado *padding* até ao comprimento máximo do *batch*. O comprimento médio das legendas é de 105 *tokens*, com um máximo de 193.

C. Divisão Treino/Teste

O *dataset* foi dividido de forma estratificada por cada filme, garantindo que a proporção de *frames* de cada filme se mantém equivalente nas duas partições: 80% para treino (448) e 20% para teste (112). Não foi utilizado um conjunto de validação separado.

IV. IMPLEMENTAÇÃO

A. Encoder

Foram testados vários *encoders* visuais ao longo do projeto. Inicialmente, utilizou-se uma CNN treinada de raiz, semelhante à desenvolvida nas aulas práticas, com variantes de canais (3, 32, 64, 128, 256) e com *Dropout* (`nn.Dropout2d(0.1)`), que não trouxe benefícios. A CNN revelou-se insuficiente para a complexidade dos *frames* dos filmes do *dataset*.

O BLIP também foi experimentado, mas o seu peso computacional tornou impossível o treino nas máquinas dos membros do grupo, mesmo recorrendo ao Google Colab. Apenas 5 a 10 épocas eram executadas e por este motivo foi abandonado.

Para a solução final testou-se dois *encoders* CLIP pré-treinados da OpenAI, ambos congelados durante todo o treino: (i) `clip-vit-base-patch16` (CLIP-16), processa imagens 224×224 em *patches* de 16×16, produzindo 197 *tokens* visuais de dimensão 768; e (ii) `clip-vit-base-patch32` (CLIP-32), processa *patches* de 32×32, produzindo 50 *tokens* visuais, com o mesmo tamanho no output.

Contudo, o *encoder* escolhido foi o CLIP-16 e é utilizado para apenas as suas *features* serem usadas como input do projetor.

A desvantagem da utilização deste *encoder* é que o CLIP-16 é mais pesado que o CLIP-32 (que embora os seus resultados fossem piores face aos do escolhido, também mostrou bons resultados), logo o treino com ele é mais lento.

A escolha de *encoders* pré-treinados justifica-se pela dimensão reduzida do *dataset* (448 amostras de treino), que tornaria o treino de raiz inviável para um *encoder* visual com capacidade suficiente para representar cenas cinematográficas complexas.

Além do referido acima, esta abordagem também tomou como referência o trabalho desenvolvido nas aulas práticas.

B. Extração das Features Visuais

O output do CLIPVisionModel é o `last_hidden_state` com *shape* (B, 197, 768) para o ViT-B/16, 197 *tokens* visuais (196 *patches* de 16×16 + 1 *token* CLS (classify token)) de dimensão 768, onde B é o *batch size* de extração. Após concatenar-se todos os *batches*, os tensores finais têm *shape* (448, 197, 768) no treino e (112, 197, 768) no teste.

Durante o treino do projetor, estes tensores são consumidos em *batches* de tamanho 8, pelo que o B efetivo em cada iteração do *loop* de treino é 8, ou seja, cada passo processa simultaneamente 1576 *tokens* visuais (8 × 197).

C. Projetor

O módulo projetor é o único componente treinável da *pipeline*. A sua função é traduzir o espaço de *embeddings* do CLIP (dimensão 768) para o espaço de *embeddings* do TinyLlama (dimensão 2048), o que permite que o LLM interprete os *tokens* visuais como se fossem *tokens* de texto.

Foram testadas duas arquiteturas:

- **Projetor Linear:** uma única camada `nn.Linear(768, 2048)`, que mapeia diretamente o espaço de representação do CLIP para o espaço de *embeddings* do TinyLlama. Esta abordagem é simples, feita com base na versão das aulas, e apresentou piores resultados em relação ao outro projetor testado. Parâmetros treináveis: $768 \times 2048 + 2048 = 1.57M$ (aproximadamente).
- **Projetor MLP:** rede de duas camadas com ativação GELU (`nn.Linear(768, 1024) → nn.GELU() → nn.Linear(1024, 2048)`). A dimensão intermédia de 1024 permite uma projeção não-linear o que ajuda a aprender mapeamentos mais complexos entre os dois espaços de *embeddings*.

A motivação para explorar o MLP resulta do facto de a projeção linear poder não capturar as não-linearidades necessárias para alinhar as representações visuais do CLIP com o espaço semântico do LLM. Os resultados (Tabela ??) confirmam que o MLP melhora o BLEU-4 em cerca de 14% face ao projetor linear, no caso do CLIP-16.

D. Language Model

O *language model* utilizado é o TinyLlama [6], um modelo com 1.1 biliões de parâmetros e dimensão de *embedding* oculta

de 2048. O modelo foi mantido congelado durante todo o treino.

As adaptações necessárias foram: (i) utilização do *tokenizer* nativo para codificar as legendas; (ii) concatenação dos *tokens* visuais projetados com os *embeddings* das legendas na dimensão temporal antes de entrar no modelo; (iii) definição de *labels* com -100 nas posições dos *tokens* visuais, de modo a que a *cross-entropy loss* seja calculada apenas sobre os *tokens* da legenda; (iv) uso de *float16* para o LLM e *float32* para o projetor, com uma conversão explícita.

E. Treino

Os hiperparâmetros utilizados no treino foram os seguintes:

- 50 épocas (suficientes para convergência com 448 amostras);
- *batch size* de 8 (equilíbrio entre VRAM e o resultado obtido, uma vez que *batch size*=16 se revelou demasiado exigente);
- otimizador AdamW com *learning rate* de 1×10^{-4} (o AdamW foi preferido face ao Adam por aplicar *weight decay* de forma correta, contribuindo para a regularização do projetor);
- resolução da imagem de entrada de 224×224 pixels (resolução padrão do *encoder* openai/clip-vit-base-patch16);
- dimensão da camada oculta do projetor de 1024 (valor intermédio entre a dimensão de saída do CLIP (768) e a dimensão de entrada do LLM (2048)).

Não foi utilizado *scheduler* de *learning rate* e apenas os parâmetros do projetor foram otimizados.

A *loss* de treino é a *cross-entropy* sobre os próximos *tokens* da legenda, condicionada pelos tokens visuais. Os tokens visuais (197 posições) são concatenados com os *embeddings* da legenda antes de serem passados ao LLM, e apenas as posições correspondentes à legendas contribuem para o gradiente.

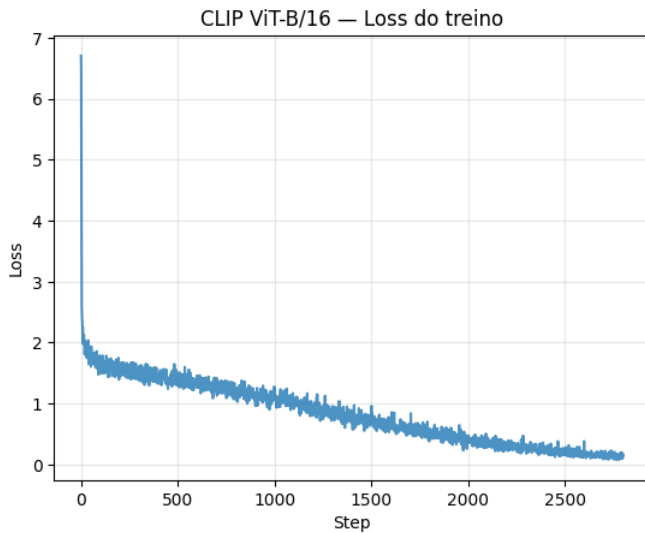


Fig. 1. Gráfico com a *loss* do CLIP ViT-B/16

F. Métricas

Foram utilizadas três métricas de avaliação automática: BLEU-4, ROUGE-1 e ROUGE-L. [7] [8]

O BLEU-4 (*Bilingual Evaluation Understudy*, n-grama de ordem 4) mede a precisão de n-gramas da predição em relação à referência, sendo uma métrica padrão em tarefas de geração de texto.

O ROUGE-1 e o ROUGE-L (*Recall-Oriented Understudy for Gisting Evaluation*, n-gramas) medem a cobertura de uni-gramas e a maior subsequência comum, respetivamente, e são particularmente relevantes para avaliar a cobertura semântica das descrições geradas.

A *loss* de treino não foi considerada métrica final, pois reflete apenas a capacidade de prever o próximo *token* da legenda de referência, não capturando a qualidade da geração livre.

G. Resultados Quantitativos

TABLE I
RESULTADOS COMPARATIVOS NO CONJUNTO DE TESTE (112 AMOSTRAS)
EM PORCENTAGEM

Configuração	BLEU-4	ROUGE-1	ROUGE-L
CLIP-16 + Linear	7.49	41.00	27.90
CLIP-16 + MLP	7.85	41.51	28.01
CLIP-32 + Linear	6.69	40.78	27.72
CLIP-32 + MLP	7.62	38.93	26.94

A melhor configuração global é o CLIP-16 com projetor MLP, que atinge BLEU-4 de 7.85% e ROUGE-1 de 41.51%. O CLIP-16 supera consistentemente o CLIP-32, o que é consistente com o facto de *patches* mais pequenos capturarem mais detalhe espacial, o que é relevante para cenas com múltiplos elementos. O projetor MLP melhora o BLEU-4 em ambos os *encoders*, o que indica que a não-linearidade adicional é benéfica para o alinhamento das representações.

H. Análise Qualitativa

As melhores predições (ROUGE-1 acima de 55.5%) evidenciam que o modelo consegue identificar corretamente a cena geral (ambiente da imagem, cena de ação, cena animada) e alguns detalhes visuais (vestuário, iluminação, presença de múltiplos personagens).

Por exemplo, no índice 89 (filme *The Fall Guy*), o modelo descreve corretamente a personagem, a ação que ela executa e a iluminação da cena.

A verdadeira legenda do *dataset* e o resultado da *pipeline* para esta imagem foram os seguintes:

True: The image depicts a close-up scene where a woman with blonde hair and blue eyes is engaged in a serious conversation with a man whose face is partially visible. The woman appears to be expressing concern or urgency, while the man, dressed in a yellow shirt, listens attentively. The setting seems to be indoors, with warm

lighting that creates a intimate atmosphere.

CLIP: The image depicts a woman with long, blonde hair, engaged in a conversation with a partner on a street corner. The woman appears to be focused and engaged, with a casual, yet intimate atmosphere. The lighting and the expression of the partner indicate a serious or emotional conversation, with the setting adding to the overall tone and significance of the scene.



Fig. 2. Imagem no índice 89 do filme *The Fall Guy*

Os casos menos bem-sucedidos apresentam alucinações, como atribuir elementos inexistentes à cena, ou a menção de objetos.

Por exemplo, no índice 98 (filme *The Fall Guy*), o modelo descreve a personagem como sendo um sobrevivente e admirado com o que o rodeia e também descreve o cenário como sendo uma paisagem com vegetação, o que não corresponde à realidade.

A verdadeira legenda do *dataset* e o resultado da *pipeline* para esta imagem foram os seguintes:

True: The image depicts a person lying down, wearing an oxygen mask and strapped to a red wooden frame. The setting appears to be a medical or emergency situation, suggesting that the individual may be in need of medical assistance.

CLIP:), or the survivor, the subject is cast in a dynamic position with a blurred or distant background, suggesting a situation of ongoing interaction or concern. The expression on their face is variably contemplative or curious, likely indicating a central role in the scenario. The background features a variety of vegetation and people, suggesting a natural setting or a background for a group activity. Compared to the person in the front position, the individual in the back appears to be in a more vulnerable or

dependent position, suggesting a relationship of support or care.



Fig. 3. Imagem no índice 89 do filme *The Fall Guy*

I. Limitações

A principal limitação do sistema é a dimensão reduzida do *dataset* de treino (448 amostras), o que é insuficiente para o modelo aprender a alinhar representações visuais e linguísticas de forma robusta.

A ausência de *fine-tuning* do LLM é outra limitação, uma vez que o TinyLlama não foi adaptado especificamente para *captioning* de cenas cinematográficas. Finalmente, a geração por *greedy decoding* (`do_sample=False`) pode produzir legendas repetitivas e menos diversas. Isto, porque o modelo, neste modo, escolhe o *token* com a maior probabilidade como próximo *token* a ser gerado.

V. CONCLUSÃO

Este trabalho apresentou um sistema de *image captioning* para *frames* de filmes baseado numa *pipeline Encoder*, Projetor e LLM.

Testaram-se quatro configurações combinando dois *encoders* CLIP (`patch16` e `patch32`) com dois tipos de projetor (Linear e MLP). A melhor configuração, CLIP-16 com MLP, atingiu BLEU-4 de 7.85% e ROUGE-1 de 41.51% num conjunto de teste de 112 *frames*.

O *encoder* CLIP pré-treinado revelou-se claramente superior às alternativas treinadas de raiz, e o projetor MLP contribuiu para ganhos mais consistentes em relação à projeção linear.

VI. TRABALHO FUTURO

Com mais tempo e recursos, seria interessante explorar um conjunto de dados maior e mais diverso, incluindo cenas de outros filmes. Também poderia recorrer-se a *fine-tuning* do LLM para melhor alinhamento com a tarefa de legendar imagens. A utilização de geração com *sampling* (com ajuste de temperatura e *top-p*) seria experiência a fazer para tentar produzir legendas mais naturais e diversificadas.

Por fim, seria relevante explorar outras métricas de avaliação, nomeadamente métricas semânticas que capturem melhor a similaridade semântica entre as legendas geradas e as de referência, ultrapassando as limitações das métricas baseadas em n-gramas.

REFERÊNCIAS

- [1] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, "Show and tell: A neural image caption generator," in Proc. CVPR, 2015, pp. 3156–3164.
- [2] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, et al., "Learning transferable visual models from natural language supervision," in Proc. ICML, 2021, pp. 8748–8763.
- [3] J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation," in Proc. ICML, 2022, pp. 12888–12900.
- [4] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in Proc. NeurIPS, 2023.
- [5] A. V. Duarte, X. Zhao, A. L. Oliveira, and L. Li, "DIS-CO: Discovering Copyrighted Content in VLMs Training Data," 2025. [Online]. Available: [arXiv:2502.17358](https://arxiv.org/abs/2502.17358).
- [6] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An Open-Source Small Language Model," 2024. [Online]. Available: [arXiv:2401.02385](https://arxiv.org/abs/2401.02385).
- [7] "LLM Evaluation: Frameworks, Metrics, and Best Practices," SuperAnnotate, 2024. [Online]. Available: <https://www.superannotate.com/blog/llm-evaluation-guide>.
- [8] J. Ip, "LLM Evaluation Metrics: The Ultimate LLM Evaluation Guide," Confident AI, 2024. [Online]. Available: <https://www.confident-ai.com/blog/llm-evaluation-metrics-everything-you-need-for-llm-evaluation>.